

睡前小记忆！特别是面试阶段的同学，睡前及醒来起床前必看！！！

一天看一个，今天是define

define关键字

重要提醒：笔试的时候，宏的拼写用大写字母！虽然语法不会报错，但会留下不专业的印象。

`#define` 是 C 语言中的一个预处理指令，而不是一个编译时的关键字。

它用于在预处理阶段向源代码中定义宏。

宏是源代码中的一个标识符，被替换为另一个字符串，这在编译之前发生。

下面是关于 `#define` 的一些基本用法和细节：

1. 定义简单的宏

示例：定义一个常量

```
#include <stdio.h>

// 定义一个简单的宏
#define PI 3.14159

int main() {
    double radius = 10.0;
    double area = PI * radius * radius;
    printf("The area of the circle is: %f\n", area);
    return 0;
}
```

这个宏定义使得在代码中每次出现 `PI` 时，预处理器都会将其替换为 `3.14159`。

两个好处（面试过程中可能会问）：

- 批量修改，只要修改PI 后面的 3.14.159，那么后续代码中都会被修改，如果没有用宏定义，有多少个地方出现3.14159就要修改多少次
- 程序可阅读性更高，比如 `setArea(100,200)`; `setArea`函数假设是获取某个区域，100,200怪怪的，如果`define WIDTH 100`，`define HEIGHT 200`，那么在函数调用的时候，`setArea(WIDTH, HEIGHT)`;看着就非常直观。

2. 定义带参数的宏

！！！！！！ 示例代码高频出现笔试中！！！！！！！！！！

示例：定义求平方

```
#include <stdio.h>
// 定义一个带参数的宏
#define SQUARE(x) ((x) * (x))

int main() {
    int num = 5;
    int square = SQUARE(num);
    printf("The square of %d is %d\n", num, square);
    return 0;
}
```

这个宏定义使得 `SQUARE(a)` 会被替换为 `((a) * (a))`。

示例：定义一年多少天

```
#include <stdio.h>

// 定义一个带参数的宏，计算一年的天数
#define DAYS_IN_YEAR(isLeapYear) ((isLeapYear) ? 366 : 365)

int main() {
    int isLeapYear = 1; // 假设这是闰年
    int days = DAYS_IN_YEAR(isLeapYear);
    printf("A leap year has %d days.\n", days);

    isLeapYear = 0; // 假设这是平年
    days = DAYS_IN_YEAR(isLeapYear);
    printf("A common year has %d days.\n", days);

    return 0;
}
```

在这个代码中，`DAYS_IN_YEAR` 宏接受一个参数 `isLeapYear`，如果该参数为真（非零值），则返回366天，表示闰年；如果为假（零值），则返回365天，表示平年。

当你编译并运行这段代码时，它会输出：

```
A leap year has 366 days.
A common year has 365 days.
```

这样，你可以根据输入的参数来判断一年有多少天了。

示例：返回小值

宏可以用来替换复杂的代码表达式，简化代码的编写。

```
#define MIN(a, b) ((a) < (b) ? (a) : (b))
int min = MIN(5, 10);
```

示例：宏与函数的比较

宏定义一个加法，也经常考！

```
#include <stdio.h>

// 定义一个宏
#define ADD(a, b) ((a) + (b))

// 定义一个函数
int add(int a, int b) {
    return a + b;
}

int main() {
    int result1 = ADD(3, 4);
    int result2 = add(3, 4);
    printf("Result of macro: %d\n", result1);
    printf("Result of function: %d\n", result2);
    return 0;
}
```

在这个示例中，我们定义了一个 `ADD` 宏和一个 `add` 函数，它们都用来计算两个数的和，并在 `main` 函数中调用。

这些示例展示了 `#define` 在不同场景下的使用方式，包括简单的宏定义、带参数的宏、字符串化宏以及宏与函数的比较。

!!!! 带参数的宏与函数的区别，面试会问，能回答以下三个就够用!!!!：

- 宏在预处理阶段展开，没有类型检查，而函数调用在编译时处理，有类型检查。
- 宏可以处理多行语句，而函数不能直接返回多行语句。
- 宏的参数没有存储空间的概念，而函数的参数在调用时会占用栈空间。

下面是带参数宏定义和函数的区别，以表格形式展示：了解一下就行，主要是上面三点

特性	带参数宏定义	函数
定义	使用 <code>#define</code> 和替换文本定义宏	使用 <code>return</code> 语句和花括号 <code>{}</code> 定义函数
类型检查	没有类型检查，仅文本替换	有类型检查，参数和返回值都有明确类型
作用域	宏在预处理阶段展开，没有实际的代码作用域概念	函数有作用域，可以访问局部变量和参数
调用方式	直接使用宏名和参数，如 <code>MACRO_NAME(arg1, arg2)</code>	调用函数名和参数，如 <code>functionName(arg1, arg2)</code>
存储	不占用内存，宏展开后替换为代码	函数体存储在内存中，占用空间

特性	带参数宏定义	函数
效率	通常比函数调用更快，因为它们在编译前展开，没有调用开销	函数调用有额外的调用和返回开销
调试	调试时看到的是展开后的代码，可能难以追踪原始宏调用	调试时可以轻松定位到函数调用和函数体
副作用	宏展开可能导致意外行为，如重复计算或与运算符优先级相关的问题	函数调用不会导致这类问题
多行语句	可以定义包含多行语句的宏	函数体可以包含多行语句
返回值	宏不返回值，它们只是代码替换	函数可以返回值
内存管理	不涉及内存分配和释放	函数可以使用动态内存分配（如 <code>malloc</code> 、 <code>free</code> ）
重入性	宏可能不是重入的，因为它们可能依赖全局状态或外部变量	函数设计为重入的，不依赖于外部状态

这个表格总结了带参数宏定义和函数在定义、类型检查、作用域、调用方式、存储、效率、调试、副作用、多行语句、返回值和内存管理等方面的区别。选择使用宏还是函数，取决于具体的应用场景和对上述特性的需求。

示例：字符串化运算符：

这个出现不多

字符串化运算符（`#`）是 C 语言预处理中的一个特性，它用于将宏的参数转换成一个字符串字面量。这个运算符在宏定义中非常有用，尤其是当你需要在编译时将某个标识符或表达式转换成字符串时。

```
#include <stdio.h>

// 这个宏 STRINGIFY 接受一个参数 x，并将其转换成一个字符串。当你使用这个宏时，x 被替换为 #x，
// 预处理器会将 x 视为一个字符串。
#define STRINGIFY(x) #x

int main() {
    const char* name = "Kimi";
    // 直接打印变量的值
    printf("The name is: %s\n", name);
    // 使用宏将变量名转换为字符串，并打印
    printf("The name of the variable is: %s\n", STRINGIFY(name));
    return 0;
}
```

当你编译并运行这段代码时，它会输出：

```
The name is: Kimi
```

```
The name of the variable is: name
```

所以这是个陷阱，我们以为像`define`一个`PI`一样，会输出`3.14159`，以为这里会输出`Kimi`，其实不是!!!

在这个示例中，`STRINGIFY` 宏将 `name` 转换成了字符串 `"name"`，并在 `printf` 函数中使用。

3. 宏的取消定义 (#undef)

宏的取消定义 (`#undef`) 是一个预处理指令，用于取消之前定义的宏。这在某些情况下很有用，比如当你想要重新定义一个宏或者在条件编译中控制宏的定义。

以下是一个包含 `main` 函数的示例代码，展示如何使用 `#undef` 来取消宏的定义：

```
#include <stdio.h>

// 定义一个宏
#define CHEN "very handsome"

int main() {
    // 使用宏
    printf("%s\n", CHEN);

    // 取消宏的定义
    #undef CHEN

    // 尝试使用已取消定义的宏（这将导致编译错误）
    // printf("%s\n", GREETING);

    return 0;
}
```

在这个示例中，我们首先定义了一个名为 `CHEN` 的宏，它包含了一个字符串。在 `main` 函数中，我们使用这个宏来打印一条消息。接着，我们使用 `#undef` 指令取消了 `CHEN` 宏的定义。如果我们尝试再次使用 `CHEN` 宏，预处理器会报错，因为它已经不再定义了。

编译并运行这段代码时，输出将会是：

```
very handsome
```

4. 修饰头文件

假设我们有一个头文件 `myheader.h`，我们希望确保其中的内容只被包含一次：

```
// myheader.h
#ifndef MYHEADER_H
#define MYHEADER_H

// 头文件内容
void myFunction() {
    // 函数实现
}

#endif // MYHEADER_H
```

在这个头文件中，`#ifndef MYHEADER_H` 检查 `MYHEADER_H` 宏是否未定义，如果是，则定义它，并包含头文件的内容。`#endif` 标记了条件编译块的结束。

现在，让我们看看如何在一个 C 程序中使用这个头文件：

```
#include <stdio.h>

// 假设这是我们的头文件
#ifndef MYHEADER_H
#define MYHEADER_H

void myFunction() {
    printf("Hello from myFunction!\n");
}

#endif // MYHEADER_H

int main() {
    myFunction(); // 调用头文件中定义的函数
    return 0;
}
```

在这个程序中，当我们包含 `myheader.h` 时，`#ifndef`、`#define` 和 `#endif` 确保了即使 `myheader.h` 被多次包含（例如，通过多个不同的源文件），`myFunction` 也只会被定义一次。这样可以避免链接错误和重复定义的问题。

编译并运行这段代码时，输出将会是：

```
Hello from myFunction!
```

这种使用 `#ifndef`、`#define` 和 `#endif` 的技术被称为“包含卫士”或“头文件卫士”，是管理头文件内容被多次包含的一种标准方法。